

Fullstack Development

JS

History

- <https://roadmap.sh/guides/history-of-javascript>
- https://www.w3schools.com/js/js_versions.asp

Fun Facts

- The brand `JavaScript` is owned (trademarked) by `Oracle`.
 - The standard that governs JavaScript has to be called something else: `ECMAScript (ES)`.
 - The world's most famous JavaScript events cannot be called "JavaScript Conference" or "JavaScript Summit". (Use `JSConf` instead)



Fun Facts

- Who is maintaining JavaScript today?
 - TC39 (Technical Committee 39) ➡ ES Standard
 - Google ➡ V8 Engine
 - WC3C ➡ Web APIs
 - The OpenJS Foundation ➡ NodeJS

Where is JS?

- Browser
 - Manipulate DOM.
- Host OS
 - JavaScript runtimes ([NodeJS](#), [Bun](#), [Deno](#), ...)

Activity

- Run `console.log` in a browser
- Run `console.log` in a terminal.

JS Roadmap

- <https://roadmap.sh/javascript>
- <https://roadmap.sh/questions/javascript>



TypeScript

What is TypeScript?

- TypeScript is a super set of JavaScript.
- Developed and maintained by `Microsoft`.

Why TypeScript?

- Improves your productivity while helping avoid bugs.
 - Catch bugs at the compile-time instead of at runtime.
- Brings the future JavaScript to today.
 - You can use the new JavaScript features before web browsers (or other environments) fully support them.
- More competitive in job market.
- Release History

How it works?

1. Write TypeScript codes. (`.ts` files)
2. Compile the TypeScript codes into plain JavaScript codes (`.js` files) using a TypeScript compiler.
3. Run JavaScript codes in an environment that JavaScript runs.

Node 23+

- Node 23+ has built-in support for TypeScript. You can run TypeScript files directly using the `node` command without needing to compile them first.

Tools

- TypeScript Playground: <https://www.typescriptlang.org/play>
- VSCode Extension
 - Quokka.js / TypeScript Worksheet
 - Pretty TypeScript Errors

Demo

```
let a = 10;  
a.slice(0, 1); // See error message
```

```
let a: number;  
a = "Hello"; // See error message
```

```
let a: number;  
// Try typing "a" and trigger intellisense.
```

Defining types

- Types by inference
 - TypeScript knows the JavaScript language and will generate types for you in many cases.
- Type by specification
 - We define it ourselves.
 - Keywords `type` , `interface`

Type by inference

```
const user = {  
  name: "Hayes",  
  id: 0,  
};  
  
user.name = 20; // Error  
console.log(user.food); // Error
```

- TypeScript already knows the type of this variable.

Type by specification

```
interface User {  
  name: string;  
  id: number;  
}
```

or

```
type User = {  
  name: string;  
  id: number;  
};
```

Type Annotation

```
interface User {  
  name: string;  
  id: number;  
}  
  
// type User = {  
//   name: string;  
//   id: number;  
// };  
  
const user: User = {  
  name: "Hayes",  
  id: 0,  
  age: 30, // Error  
};
```

type vs interface

- They are very similar, and for the most common cases act the same.
- However, TypeScript doc recommends `interface`.
 - `interface` provides better error message.
 - `interface` can be extended.

More type demo

- https://github.com/fullstack-69/typescript-intro/blob/main/lecture/03_more_type_demo.ts

Type utilities

```
interface User {  
  id: number;  
  name: string;  
}  
  
// Extend  
interface UserExtended extends User {  
  isActive: boolean;  
}
```

More information

Function (argument type)

```
// Give warning
function sumNumber(a, b) {
  return a + b;
}

// OK
function sumNumber(a: number, b: number) {
  return a + b;
}
```

Function (type guards)

```
// Hover cursor on "text" to see the type
function greeter(text: string | null | undefined) {
  if (!text) {
    console.log("...");
    return;
  }
  console.log(text);
}
```

Generics

Generics provide variables to types.

```
interface Backpack<Type> {  
    items: Type[];  
    add: (obj: Type) => void;  
    get: () => Type;  
}  
  
const backpack: Backpack<string> = {  
    items: ["1", "2"],  
    add: (myStr) => {  
        myStr.slice(0, 1);  
    },  
    get: () => {  
        return "Hi";  
    },  
};
```

Use Typescript in NodeJS project

Setup

- `npm i -g pnpm` (*Alternative package manager*)
- `pnpm init`
 - Note: Unlike previous version of pnpm, `package.json` will have `"type": "module"` now.
- `pnpm install -D typescript @types/node`
- Create `./src` directory

- Create `./src/index.ts`

```
function sayHello(name: string) {  
  console.log("Hello " + name);  
}  
  
sayHello("World");
```

Compile

- `pnpm exec tsc src/index.ts --outDir dist`

Run JavaScript

- `node ./dist/index.js`

Run TypeScript file directly (node 23+)

- `node ./src/index.ts`

Note on pnpm V11

- New `devEngines` option in `package.json` will prevent the use of `npx`.

```
1  {
2    "name": "New folder",
3    "version": "1.0.0",
4    "description": "",
5    "main": "index.js",
6    "scripts": {
7      "test": "echo \"Error: no test speci-
8    },
9    "keywords": [],
10   "author": "",
11   "license": "ISC",
12   "devEngines": {
13     "packageManager": {
14       "name": "pnpm",
15       "version": "^11.9.0",
16       "onFail": "download"
17     }
18   },
19   "type": "module"
20 }
21
```

Use TS configuration file

- `pnpm exec tsc --init`
- Modify `tsconfig.json`

```
{
  // Other Outputs (Commented out)
  // "sourceMap": true,
  // "declaration": true,
  // "declarationMap": true,

  "compilerOptions": {
    "rootDir": "./src",
    "outDir": "./dist"
  },
  "include": ["./src/**/*.ts"]
}
```

Use TS configuration file (cont)

- Now just type `pnpm exec tsc`
- Use `pnpm exec tsc --showConfig` to see config.

Code - Async

```
async function getData() {  
  const res = await fetch("https://jsonplaceholder.typicode.com/todos/1");  
  return await res.json();  
}  
  
getData().then((data) => console.log(data));
```

- Try changing `"target": "es6"` and see the generated code.

Code - Import

./src/lib.ts

```
export const name = "Tom";
```

./src/index.ts

```
import { name } from "./lib.js";  
  
sayHello(name);
```

Note on import statement

```
import { name } from "./lib.js";
```

- Notice the `.js` extension in the import statement. This is required when using ES modules in NodeJS `"module": "nodenext"`.
- If you change it to `"module": "esnext"`, you can omit the extension, but it may cause issues in some environments.

Code - FS

```
import * as fs from "fs";
import { fileURLToPath } from "url";
import { dirname } from "path";
const __filename = fileURLToPath(import.meta.url);
const __dirname = dirname(__filename);

const dir = fs.readdirSync(__dirname);
console.log(dir);
```

You need to enable the following in your `tsconfig.json` to use NodeJS built-in modules:

```
"lib": ["esnext"],
"types": ["node"],
```

Using TSconfig template

- `pnpm install -D @types/node @tsconfig/node-lts @tsconfig/node-ts` (*What*)

`./tsconfig.json`

```
{
  "extends": [
    "@tsconfig/node-lts/tsconfig.json",
    "@tsconfig/node-ts/tsconfig.json"
  ],
  "compilerOptions": {
    "types": ["node"],
    "rootDir": ".",
    "outDir": "./dist"
  },
  "include": ["./src/**/*.ts"]
}
```

TypeScript - React

Installation

- `pnpm create vite@latest`
 - Choose `React` and `TypeScript` options.

TSConfig

```
{
  "files": [],
  "references": [
    { "path": "./tsconfig.app.json" },
    { "path": "./tsconfig.node.json" }
  ]
}
```

- `references` allows us to have multiple `tsconfig` files for different purposes.
 - `tsconfig.app.json` is for the React app
 - `tsconfig.node.json` is for NodeJS scripts.

./src/Card.tsx

```
import { type FC } from "react";

interface Props {
  title: string;
  text?: string;
}

const Card: FC<Props> = ({ title, text }) => {
  return (
    <div style={{ border: "1px solid gray" }}>
      <div>{title}</div>
      <div>{text ?? "..."}</div>
    </div>
  );
};

export default Card;
```

./src/App.tsx

```
import Card from "../Card";

const App = () => {
  return (
    <main>
      <Card title="Card 1" text="Text 1" />
      <Card title="Card 2" text="Text 2" />
      <Card title="Card 3" />
    </main>
  );
};

export default App;
```